

**UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA**

Savremene obrazovne tehnologije i standardi

Inteligentni Q&A agent kao IMS LTI alat sa RAG arhitekturom

*Integracija u Canvas LMS, lokalni LLM i automatska sinhronizacija nastavnih
materijala*

Boris Letić E2 126/2025

Novi Sad, 2026.

Sadržaj

Apstrakt.....	3
1. Uvod	4
1.1 Ciljevi rada.....	4
2. Teorijske osnove	4
2.1 Sistemi za upravljanje učenjem (LMS).....	4
2.2 IMS Learning Tools Interoperability standard.....	5
2.3 Verzije LTI standarda.....	5
2.4 Retrieval-Augmented Generation (RAG).....	5
3. Arhitektura sistema	7
3.1 Pregled.....	7
3.2 Komponente sistema	7
3.3 Tok podataka pri Q&A interakciji.....	7
4. LTI integracija sa Canvas LMS-om	8
4.1 LTI launch tok.....	8
4.2 Procesiranje launch zahteva.....	8
5. Canvas Files API i sinhronizacija materijala	9
5.1 Motivacija - prava LMS integracija	9
5.2 Autentifikacija - Developer Token	9
5.3 Pull mehanizam	9
5.4 Delta sinhronizacija	10
6. RAG implementacija	11
6.1 Embedding model.....	11
6.2 Vektorska baza - ChromaDB	11
6.3 Lokalni LLM - Ollama.....	12
6.4 Generisanje odgovora i confidence score	12
7. Korisnički interfejs i tok rada.....	14
8. Tehnološki stack i deployment	14
9. Zaključak.....	15
Literatura.....	16

Apstrakt

Ovaj seminarski rad opisuje projektovanje i implementaciju inteligentnog Q&A agenta koji se integriše u Canvas Learning Management System koristeći IMS Learning Tools Interoperability (LTI) standard. Sistem omogućava studentima da postavljaju pitanja o nastavnom materijalu kursa direktno iz Canvas okruženja, a odgovori se generišu primjenom RAG (Retrieval-Augmented Generation) arhitekture nad lokalnim velikim jezičkim modelom (Ollama Mistral).

Posebnu pažnju zaslužuje sinhronizacija nastavnih materijala: tool ne zahtijeva poseban upload, već automatski povlači fajlove iz Course Files sekcije Canvas kursa preko Canvas REST API-ja. Ovim se ostvaruje prava LMS integracija u kojoj nastavnik održava materijale samo na jednom mjestu. Sistem je u potpunosti deployovan kroz Docker Compose orkestraciju i koristi besplatne komponente - bez zavisnosti od plaćenih eksternih servisa.

1. Uvod

Sistemi za upravljanje učenjem (Learning Management Systems, LMS) postali su standardna infrastruktura visokog obrazovanja. Platforme poput Canvas-a, Moodle-a i Blackboard-a objedinjuju distribuciju materijala, ocjenjivanje, komunikaciju i administraciju kurseva. Međutim, sami LMS-ovi su namjerno generički - fokusirani su na kalendarsko-administrativne funkcije, dok specijalizovane pedagoške alate (interaktivne simulacije, AI tutore) prepuštaju eksternim servisima.

IMS Learning Tools Interoperability (LTI) standard rješava upravo ovaj problem: definiše protokol kojim se eksterni alati (Tool Providers) bezbjedno integrišu u LMS (Tool Consumer) uz prenos konteksta korisnika i kursa. Studenti pristupaju eksternom alatu preko jednog klika u LMS-u, bez zasebne autentifikacije.

Paralelno sa ovim, razvoj velikih jezičkih modela (LLM) otvorio je mogućnost izgradnje obrazovnih asistenata koji ne odgovaraju nasumično, već zasnovano na konkretnom nastavnom materijalu. Tehnika koja to omogućava - Retrieval-Augmented Generation (RAG) - kombinuje semantičku pretragu nad korpusom dokumenata sa generisanjem odgovora kroz LLM. Time se postiže grounding: model ne halucinira, već citira sadržaj iz materijala kursa.

Cilj ovog projekta je spojiti ova dva svijeta - LTI i RAG - u funkcionalan, deployabilan sistem koji nastavnici mogu ugraditi u svoj postojeći Canvas kurs bez izmjene radnog toka.

1.1 Ciljevi rada

- Implementirati LTI alat koji se integriše u Canvas LMS preko standardizovanog launch toka.
- Omogućiti automatsku sinhronizaciju nastavnih materijala iz Canvas Course Files preko Canvas REST API-ja.
- Izgraditi RAG pipeline (embedding, vektorska pretraga, generisanje) koristeći isključivo besplatne komponente.
- Demonstrirati cjelokupan sistem kroz Docker Compose orkestraciju koja pokriva Canvas, LTI tool, Ollama LLM, ChromaDB i prateće servise.
- Prikazati kako standardizacija (LTI) i savremene AI tehnike (RAG) zajedno čine obrazovne komponente.

2. Teorijske osnove

2.1 Sistemi za upravljanje učenjem (LMS)

LMS je integrisana platforma za administraciju, dokumentaciju, praćenje i izvještavanje obrazovnih programa. Tipičan LMS objedinjuje module za kurseve, korisnike (studente, nastavnike, administratore), zadatke, ocjene, diskusije i kalendar. Canvas, koji je razvio Instructure, je jedan od najzastupljenijih LMS-ova u akademskom okruženju zbog moderne REST/GraphQL API podrške, otvorenog koda i pouzdane LTI implementacije.

Karakteristika modernog LMS-a koja je relevantna za ovaj projekat je tzv. extensibility model: LMS sam ne pokušava da bude sve, već pruža interfejsse za uključivanje eksternih alata. Ti

interfejsi su dvostruki - REST API (za programski pristup podacima) i LTI (za interaktivnu integraciju alata u korisničko iskustvo).

2.2 IMS Learning Tools Interoperability standard

LTI je W3C-style standard koji definiše IMS Global konzorcijum. Cilj standarda je da omogući eksternim alatima da se ponašaju kao prirodni dio LMS-a uz minimalan integration overhead. Tri ključna koncepta:

- Tool Consumer - sistem koji "konzumira" alate, tipično LMS (Canvas, Moodle).
- Tool Provider - eksterni servis koji pruža funkcionalnost (u našem slučaju, Q&A agent).
- Launch - proces u kojem korisnik iz LMS-a klikne na alat, a LMS proslijeđuje potpisani zahtjev tool provider-u sa kontekstom (ko je korisnik, koji je kurs, koja je uloga).

Ključna prednost LTI-ja je single sign-on: student se autentifikuje samo jednom u LMS, a svi LTI alati prihvataju njegov identitet bez dodatne prijave.

2.3 Verzije LTI standarda

Standard je evoluirao kroz više verzija, svaka sa drugačijim sigurnosnim modelom:

Verzija	Autentifikacija	Status
LTI 1.0	OAuth 1.0 (HMAC-SHA1)	Zastarjela
LTI 1.1	OAuth 1.0 sa proširenjima	Široko podržana, korištena u projektu
LTI 1.3 / Advantage	OAuth 2.0 + OpenID Connect + JWT	Aktuelna preporuka

Iako je LTI 1.3 formalno preporučena verzija, Canvas u njegovoj stable Docker distribuciji ima ograničenu podršku za 1.3 - kreiranje LTI 1.3 Developer Key-eva preko web interfejsa ima nepouzdan UI. U ovom projektu zato koristimo LTI 1.1, koji je funkcionalno dovoljan za zahtijevanu integraciju i radi pouzdano. Migracija na 1.3 svodi se na zamjenu OAuth 1.0 potpisivanja JWT-om.

2.4 Retrieval-Augmented Generation (RAG)

RAG je arhitektonski obrazac uveden 2020. godine (Lewis et al.) koji rješava centralni problem velikih jezičkih modela - halucinacije i zastarjelost znanja. Umjesto da se model pita direktno, prvo se nad korpusom relevantnih dokumenata izvrši semantička pretraga, a zatim se pronađeni odlomci ubace u prompt zajedno sa pitanjem. Time se model navodi da generiše odgovor zasnovan na konkretnom kontekstu, a ne na svojoj trening distribuciji.

RAG pipeline ima dvije faze:

- Indeksiranje (offline): dokumenti se cijepaju na chunkove, svaki chunk se kodira u vektor pomoću embedding modela, vektori se čuvaju u vektorskoj bazi.

- Upit (online): korisničko pitanje se kodira istim embedding modelom, traže se top-K najbližih chunkova po cosine sličnosti, ti chunkovi se ubacuju u prompt LLM-a koji generiše finalni odgovor.

U obrazovnom kontekstu, RAG je posebno koristan jer omogućava verifikaciju izvora - svaki odgovor se može pratiti do konkretnog dijela materijala iz kursa. Ovo je fundamentalno drugačije od korištenja čistog ChatGPT-a, koji može dati fluentan ali nereferensiran odgovor.

3. Arhitektura sistema

3.1 Pregled

Sistem se sastoji od pet glavnih servisa, orkestriranih kroz Docker Compose. Svaki servis je izolovan kontejner sa jasno definisanom odgovornošću:

Servis	Tehnologija	Uloga
Canvas LMS	Instructure Canvas (stable)	Tool Consumer, hostuje kurseve i materijale
LTI Q&A Tool	Flask + Gunicorn (Python)	Tool Provider, glavna aplikativna logika
Ollama	Ollama runtime (Mistral 7B)	Lokalni LLM server za generisanje odgovora
ChromaDB	Chroma vector DB	Vektorska baza za embedding indeksiranje
PostgreSQL + Redis	PostgreSQL 11, Redis 7	Storage backend za Canvas

3.2 Komponente sistema

Tri komponente nose centralnu logiku LTI tool-a:

- LTI Handler - implementira launch endpoint koji prima OAuth-potpisani POST od Canvas-a, validira potpis, ekstrahuje kontekst (user, course, role) i otvara web interfejs.
- Canvas Sync Manager - koristi Canvas REST API da povuče fajlove iz Course Files, ekstrahuje tekst (PDF, DOCX, TXT, MD) i prebaci ih u ChromaDB indeks.
- RAG Engine - odgovoran za embedding upita, retrieval relevantnih chunkova i poziv Ollama API-ja sa konstruisanim promptom.

Razdvajanje odgovornosti omogućava nezavisno testiranje i skaliranje.

3.3 Tok podataka pri Q&A interakciji

Sa stanovišta studenta, interakcija se svodi na tri koraka: otvaranje kursa, klik na Q&A asistenta, postavljanje pitanja. Iza scene, sistem prolazi sljedeći put:

- Student klikne na Q&A asistenta u Canvas modulu - Canvas šalje LTI launch POST tool-u.
- Tool validira potpis, otvara web sesiju i prikazuje chat interfejs.
- Student kuca pitanje, klijent šalje JSON na /api/ask endpoint.
- RAG engine kodira pitanje u vektor i traži top-K chunkova u ChromaDB collection-u za taj kurs.
- Ollama dobija prompt sa pitanjem i kontekstom, vraća generisani odgovor.
- Tool računa confidence score na osnovu cosine distance i vraća JSON sa odgovorom, izvorima i pouzdanošću.
- UI prikazuje odgovor sa badge-om pouzdanosti i listom citiranih chunkova.

4. LTI integracija sa Canvas LMS-om

4.1 LTI launch tok

LTI 1.1 launch je OAuth 1.0 potpisani POST zahtjev. Canvas, kada korisnik klikne na external tool, generiše skup parametara koji opisuju kontekst, potpisuje ih HMAC-SHA1 metodom korišćenjem dijeljene tajne (shared secret), i šalje ih tool-u kao form-encoded POST. Najvažniji parametri uključuju:

- user_id i lis_person_name_full - identitet i ime korisnika.
- custom_canvas_course_id i context_title - ID i naziv kursa iz kojeg dolazi launch.
- roles - ULoga (Instructor, Student, Administrator) u kontekstu kursa.
- oauth_signature i prateći OAuth parametri - za verifikaciju autentičnosti.

Tool mora da verifikuje potpis koristeći isti shared secret i da odbije zahtjev ako se potpis ne poklapa. Ovim se sprečava da bilo ko sa znanjem URL-a tool-a fingirka launch.

4.2 Procesiranje launch zahteva

U Flask aplikaciji, launch endpoint je implementiran kao standardni POST handler. Nakon validacije potpisa, ekstraktuju se parametri i pohranjuju u Flask session, što omogućava da naredne API rute (npr. /api/ask) znaju iz kojeg kursa dolaze pitanja.

```
@app.route('/launch', methods=['POST'])
def launch():
    user_id = request.form.get('user_id', 'unknown')
    user_name = request.form.get('lis_person_name_full', 'Student')

    course_id = (
        request.form.get('custom_canvas_course_id')
        or request.form.get('context_id', 'default')
    )
    course_title = request.form.get('context_title', 'Unknown Course')

    roles = request.form.get('roles', '').split(',')
    is_instructor = any('Instructor' in r or 'Teacher' in r for r in roles)

    session['user_id'] = user_id
    session['course_id'] = course_id
    session['is_instructor'] = is_instructor

    return render_template(
        'qa_interface.html',
        user_name=user_name,
        course_title=course_title,
        course_id=course_id,
        is_instructor=is_instructor,
    )
```

Zapaziti da je session['course_id'] kasnije osnova za izolaciju materijala - svaki kurs ima svoju ChromaDB collection, pa pitanja iz kursa A nikad ne dohvataju materijale iz kursa B. Multi-tenancy se postiže prirodno preko ovog ključa.

5. Canvas Files API i sinhronizacija materijala

5.1 Motivacija - prava LMS integracija

U inicijalnoj verziji projekta, tool je imao sopstveni upload widget - nastavnik je morao ručno da prebaci fajlove u tool, nezavisno od Canvas-a. Ovo je tehnički radilo, ali je konceptualno bilo pogrešno: LTI alat koji ne komunicira sa LMS-om za podatke nije stvarno integrisan, već samo embeddovan.

Prava LTI integracija znači da nastavnik radi isključivo u svom uobičajenom okruženju (Canvas Course Files), a tool se sam stara o tome da te materijale učini dostupnim Q&A engine-u. To eliminiše dvostruki radni tok i osigurava da je sadržaj koji student vidi u Files i sadržaj koji asistent koristi za odgovore - uvijek isti.

5.2 Autentifikacija - Developer Token

Canvas REST API zahtijeva autentifikaciju. Postoji više opcija - OAuth 2.0 flow, LTI Advantage Names and Role Provisioning Service, i Developer Token (Personal Access Token). Za potrebe demo-a izabran je Developer Token zbog jednostavnosti: nastavnik ga generiše ručno u Canvas Settings → Approved Integrations → New Access Token, i token se postavlja kao environment varijabla u tool-u.

Token nosi iste privilegije kao korisnik koji ga je generisao. Ako je generisan od strane nastavnika sa pristupom kursu, tool može čitati sve fajlove u tom kursu. Token se prenosi kao Bearer header u svakom HTTP zahtjevu prema Canvas-u:

```
self.session.headers.update({
    'Authorization': f'Bearer {self.api_token}',
    'Accept': 'application/json',
})
```

5.3 Pull mehanizam

Listanje fajlova u kursu obavlja se preko endpoint-a GET /api/v1/courses/:id/files. Canvas paginira odgovor preko Link header-a (RFC 5988); sync mora da prati 'next' link dok ne iscrpi sve stranice.

```
def list_course_files(self, course_id: str) -> List[Dict]:
    path = f'/api/v1/courses/{course_id}/files'
    return self._paginated_get(path)

def _paginated_get(self, path, params=None):
    params = dict(params or {})
    params.setdefault('per_page', 100)
    results = []
    url = urljoin(self.base_url + '/', path.lstrip('/'))
    while url:
        resp = self.session.get(url, params=params, timeout=30)
        resp.raise_for_status()
        results.extend(resp.json())
        params = None # next URL vec sadrzi params
        url = self._next_link(resp)
    return results
```

Svaki vraćeni fajl-objekat sadrži pre-signed download URL koji autorizuje preuzimanje kroz verifier query parametar. Tool taj URL koristi za skidanje sadržaja, ekstrahuje tekst pomoću PyPDF2 (za PDF), python-docx (za DOCX) ili direktnim čitanjem (za TXT/MD), i proslijeđuje tekst RAG engine-u na indeksiranje.

5.4 Delta sinhronizacija

Naivni pristup bi pri svakom sync-u skinuo i reindeksirao sve fajlove. To je neefikasno i nepotrebno - fajlovi se mijenjaju rijetko, a embedding generisanje je računarski skupo. Zato sistem održava tracking state na disku, mapu Canvas file ID-jeva u njihove updated_at timestamp-ove. Pri svakom sync-u uporedi se trenutno stanje u Canvas-u sa tracked stanjem; samo dodati ili promijenjeni fajlovi se procesiraju, a fajlovi koji su obrisani u Canvas-u se uklanjaju iz indeksa.

Ovo daje sistemu nekoliko korisnih osobina:

- Idempotentnost - pokretanje sync-a više puta ne mijenja krajnji rezultat ako se Canvas u međuvremenu nije promijenio.
- Performanse - tipičan re-sync je trenutno ako nema promjena, jer se Canvas API zove samo za listanje.
- Konzistentnost - fajlovi obrisani u Canvas-u nestaju iz indeksa, sprečavajući da Q&A asistent navodi materijale koji više ne postoje.

Sinhronizacija se pokreće na dva načina: automatski pri prvom otvaranju tool-a u datom kursu (kao deo LTI launch handler-a), i manuelno preko dugmeta 'Sinhronizuj sada' u instruktor panelu.

6. RAG implementacija

6.1 Embedding model

Embedding je vektorska reprezentacija teksta gdje semantički slični tekstovi imaju blizu vektore u prostoru. Kvalitet embedding modela direktno određuje kvalitet retrieval-a u RAG sistemu. Za ovaj projekat odabran je paraphrase-multilingual-MiniLM-L12-v2 iz biblioteke Sentence Transformers.

Razlozi izbora ovog modela:

- Multilingual - model je treniran na više od 50 jezika uključujući srpski, što je kritično za projekat na srpskom jeziku.
- Veličina - sa 118M parametara model staje u manje od 500MB i radi na CPU bez GPU-a.
- Brzina - embedovanje jednog chunka traje oko 30ms na običnom desktop CPU-u.
- Besplatan - distribuiran pod Apache 2.0 licencom, bez API quota.

```
from sentence_transformers import SentenceTransformer

self.embedder = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# Embedovanje upita
question_embedding = self.embedder.encode(question).tolist()
```

6.2 Vektorska baza - ChromaDB

Vektorska baza čuva embedding-e i podržava efikasnu pretragu po sličnosti. Za ovaj projekat odabran je ChromaDB - open source vektorska baza dizajnirana specifično za RAG aplikacije. Karakteristike koje su uticale na izbor:

- HNSW (Hierarchical Navigable Small World) indeks - sub-linearna pretraga čak i sa stotinama hiljada vektora.
- Cosine similarity - odgovara prirodi rečenica-embedding-a koje smo izabrali.
- Per-collection izolacija - svaki kurs ima svoj namespace (course_<id>), što sprečava unakrsno curenje sadržaja.
- Persistent storage - indeks preživljava restart kontejnera.

```
self.collection = self.chroma_client.get_or_create_collection(
    name=f"course_{course_id}",
    metadata={"hnsw:space": "cosine"}
)
# Dodavanje chunk-a
self.collection.add(
    ids=[chunk_id],
    embeddings=[embedding],
    documents=[chunk_text],
    metadatas=[{'filename': fname, 'source': 'canvas'}]
)
# Pretraga
results = self.collection.query(
    query_embeddings=[question_embedding],
    n_results=8
)
```

6.3 Lokalni LLM - Ollama

Generisanje odgovora obavlja Mistral model preko Ollama runtime-a. Ollama je open source alat koji izlaže lokalni LLM kao HTTP API, sa potpunim ekvivalentom OpenAI Chat API ergonomije. Time se izbjegava zavisnost od plaćenih servisa - tool može da radi potpuno offline.

Mistral 7B je odabran kao kompromis između kvaliteta i resursa: na sistemu sa NVIDIA RTX 4060 GPU-om, inference traje 6-10 sekundi po odgovoru. Bez GPU-a, isti odgovor traje 45-60 sekundi, što je još uvijek prihvatljivo za demo, ali GPU akceleracija značajno poboljšava korisničko iskustvo.

```
response = requests.post(
    f"{self.ollama_host}/api/generate",
    json={
        "model": "mistral",
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": 0.3,    # niska t za faktickkije odgovore
            "top_p": 0.9,
            "num_predict": 512
        }
    },
    timeout=120
)
```

6.4 Generisanje odgovora i confidence score

Prompt engineering ima ključnu ulogu u RAG sistemu. Loš prompt navodi model da halucinira ili da preformuliše svoje opšte znanje umjesto da citira kontekst. Korišteni prompt template eksplicitno instruiše model da odgovara isključivo na osnovu pruženog konteksta i da prizna nepostojanje informacije:

```
prompt = f"""Ti si obrazovni asistent. Tvoj zadatak je da odgovoris na pitanje
ISKLUJUCIVO na osnovu datog konteksta.

PRAVILA:
- Odgovori SAMO na osnovu informacija iz konteksta ispod
- Ako informacija NIJE u kontekstu, reci kratko da potrebna informacija
  nije pronadena u materijalima. Ne nagadjaj i ne izmislijaj.
- Odgovaraj NA SRPSKOM JEZIKU
- Budi precizan i koncizan

KONTEKST IZ NASTAVNIH MATERIJALA:
{context}

PITANJE STUDENTA: {question}

ODGOVOR (samo na osnovu konteksta iznad):"""
```

Confidence score se izvodi iz prosječne cosine distance pronađenih chunkova. Mali distance znači da su chunkovi semantički bliski pitanju, što indicira da je model imao kvalitetan kontekst. Distance se mapira u [0,1] raspon kroz step funkciju sa pet pragova (0.95 / 0.85 /

0.75 / 0.50 / 0.35), kalibrisanih empirijski tako da score iznad 70% odgovara odgovoru u kojem se nalazi tačan podatak iz materijala.

Ovaj score nije proizvoljna mjera - on je formalna metrika izvedena iz semantičke sličnosti, što znači da student može da procijeni pouzdanost odgovora bez čitanja izvora. Niski score je korisni signal: 'asistent nije baš siguran, provjeri original'.

7. Korisnički interfejs i tok rada

Web UI je implementiran kao single-page Tailwind CSS aplikacija unutar Jinja2 templata. Cijela interakcija je realizovana kroz JavaScript fetch pozive ka Flask backend-u, bez frontend framework-a. Razlog je jednostavnost - UI je sekundaran u odnosu na funkcionalnost tool-a, a nepotrebne zavisnosti komplikuju deployment.

Interfejs ima dva moda:

- Studentski mod - chat sa pitanjima, odgovorima, badge-om pouzdanosti i listom citiranih izvora. Ovo je default.
- Instruktorski mod - sve od studentskog, plus panel za sinhronizaciju (dugmad 'Sinhronizuj sada', 'Force re-sync', 'Pregled materijala', 'Test Canvas konekcije').

Razlika između modova je određena role parametrom iz LTI launch-a, pa nije moguće zaobići kontrolu pristupa modifikovanjem JavaScript koda - backend takođe verifikuje is_instructor flag iz session-a pri svakom administrativnom pozivu.

Poruke u chatu se prikazuju sa animacijom (fadeIn), confidence se kodira bojama (zeleno iznad 80%, žuto iznad 60%, crveno ispod), a izvori se prikazuju kao truncirani snippeti sa nazivom Canvas fajla. Loading indikator (animirane tačkice 'AI razmišlja...') daje korisniku osjećaj responsivnosti dok Ollama generiše odgovor.

8. Tehnološki stack i deployment

Cijeli sistem je orkestriran kroz Docker Compose, što znači da se deployment svodi na klonovanje repozitorijuma, postavljanje environment varijabli i pokretanje docker-compose up. Sljedeća tabela rezimira ključne tehnologije:

Sloj	Tehnologija	Verzija
LMS	Canvas (Instructure)	stable
LMS Storage	PostgreSQL / Redis	11 / 7
LTI Tool runtime	Python + Flask + Gunicorn	3.11 / 3.0 / 21.2
LTI biblioteka	PyLTI1p3	2.0
Embedding	Sentence Transformers (MiniLM)	2.2
Vector DB	ChromaDB	0.4
LLM runtime	Ollama (Mistral 7B)	latest
File parsing	PyPDF2 + python-docx	3.0 / 1.1
Frontend	Tailwind CSS + vanilla JS	via CDN
Orkestracija	Docker Compose	v2

Preduslovi za deployment su: bilo koji sistem sa Docker-om i 16GB RAM-a (ili više ako se Mistral model pokreće na CPU). GPU akceleracija je opcionalna ali preporučena za produkcijsko korišćenje sa više istovremenih korisnika.

Cjelokupna infrastruktura je besplatna i open source - projekat ne zavisi ni od jednog plaćenog SaaS servisa. Ovo je posebno značajno u akademskom kontekstu, gdje budžeti često ne dopuštaju OpenAI API kredite po studentu.

9. Zaključak

Ovaj projekat demonstrira da se savremene obrazovne tehnologije i AI tehnike mogu kombinovati u funkcionalan, deployabilan sistem koristeći isključivo standardizovane protokole i otvorene komponente. IMS LTI standard čini tool prenosivim - ista implementacija koja radi sa Canvas-om radila bi i sa Moodle-om, Blackboard-om ili bilo kojim drugim LTI-compliant LMS-om uz minimalna podešavanja.

Ključna metodološka odluka u projektu bila je odluka o izvoru nastavnih materijala. Inicijalna verzija sa direktnim upload-om u tool je tehnički bila funkcionalna, ali je prevazilazila samu suštinu LTI integracije - alat koji ne dijeli podatke sa LMS-om nije stvarno integrisan, već samo embeddovan. Migracija na Canvas Files API kroz automatsku sinhronizaciju riješila je ovaj problem i učinila tool prirodnim produžetkom Canvas-a, a ne paralelnim sistemom.

RAG arhitektura je pokazala svoje prednosti u obrazovnom kontekstu: odgovori su grounded u stvarnom materijalu kursa, confidence score daje studentu signal o pouzdanosti, a citirani izvori omogućavaju verifikaciju. Lokalni LLM (Ollama) eliminiše zavisnost od plaćenih API-ja i osigurava privatnost - pitanja studenata ne napuštaju infrastrukturu institucije.

Buduća unapređenja mogla bi obuhvatiti: migraciju na LTI 1.3 sa OAuth 2.0 i JWT-om, implementaciju Names and Role Provisioning Service za automatsko sinhronizovanje spiska studenata, integraciju Assignment and Grade Services za vraćanje rezultata Q&A interakcije nazad u Canvas Gradebook, i fine-tuning embedding modela na konkretan domen kursa za bolji retrieval.

Literatura

- [1] IMS Global Learning Consortium. (2023). Learning Tools Interoperability Core Specification v1.3. <https://www.imsglobal.org/spec/lti/v1p3/>
- [2] IMS Global Learning Consortium. (2014). Learning Tools Interoperability Implementation Guide v1.1. <https://www.imsglobal.org/specs/ltiv1p1>
- [3] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. <https://arxiv.org/abs/2005.11401>
- [4] Instructure. Canvas LMS REST API Documentation. <https://canvas.instructure.com/doc/api/>
- [5] Reimers, N., Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
- [6] Chroma. (2024). ChromaDB - the AI-native open-source embedding database. <https://www.trychroma.com/>
- [7] Ollama. (2024). Get up and running with large language models, locally. <https://ollama.com/>
- [8] Jiang, A. Q. et al. (2023). Mistral 7B. <https://arxiv.org/abs/2310.06825>
- [9] Grinberg, M. (2018). Flask Web Development. O'Reilly Media.
- [10] Docker Inc. Docker Compose specification. <https://docs.docker.com/compose/>